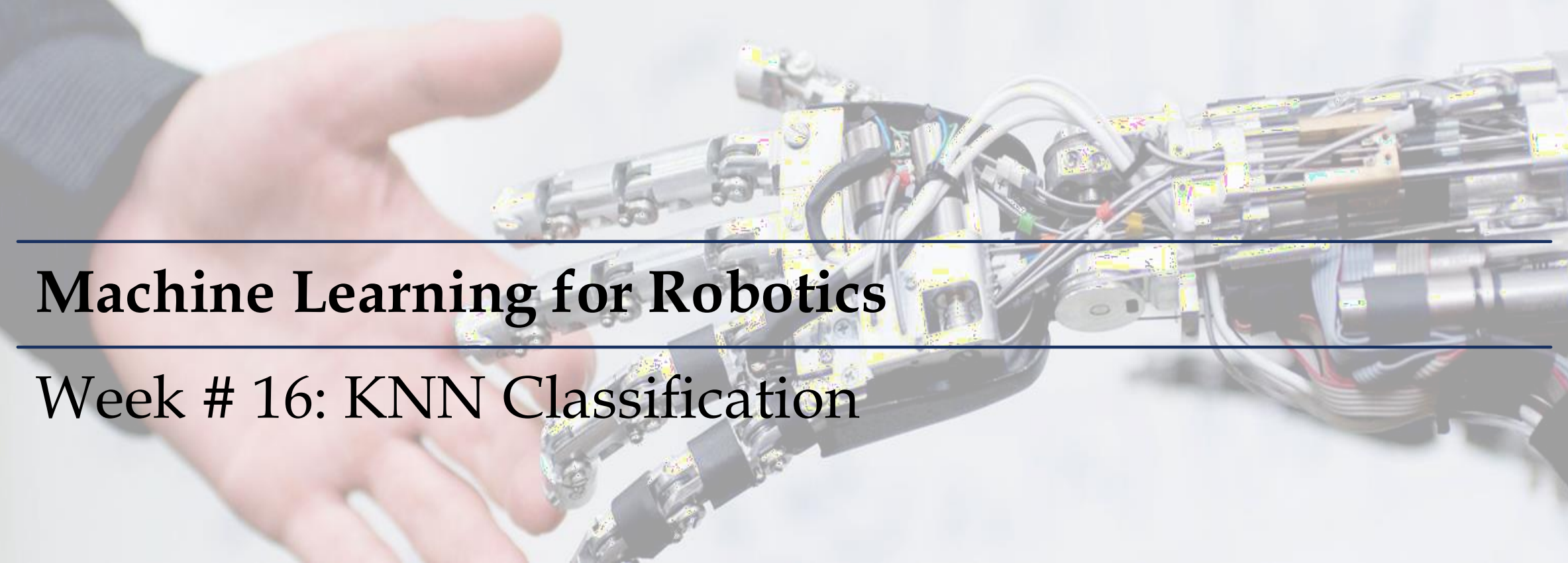


A photograph of a human hand holding a small, intricate robotic arm. The arm is constructed from various mechanical parts, including metal rods, gears, and a dense network of colored wires (white, blue, red, yellow). The background is a plain, light-colored surface.

Machine Learning for Robotics

Week # 16: KNN Classification

Dr. Maryam Iqbal

ABOUT INSTRUCTOR

- PhD EE (Controls, Automation & Rehabilitation Robotics)
- Research areas: Robotics & Automation, Artificial Intelligence, Machine Learning, Biomechanics, AI in Healthcare
- In charge : BSRIS Program at Bahria University
- Research Publications, Funded Projects and Patents
- Member of AI Task Force (Ministry of Planning, Development & Special Initiatives of Pakistan)
- Senior Member IEEE (Region 10) | General Secretary IEEE RAS (Region 3) | Member IEEE CSS
- Email: maryamiqbal.buic@bahria.edu.pk
- LinkedIn: [linkedin.com/in/maryam-iqbal-3b942a66](https://www.linkedin.com/in/maryam-iqbal-3b942a66)
- Google Scholar: <https://scholar.google.com/citations?user=-4togNoAAAAJ&hl=en>

Classification

Classification models

KNN Classification

KNN Major Assumptions

KNN Classification intuition

Python Implementation

Conclusion

CONTENTS

CLASSIFICATION



Unlike regression where you predict a continuous number, you use classification to predict a category.



There is a wide variety of classification applications from medicine to marketing.



Classification models include linear models like Logistic Regression, SVM, and nonlinear ones like K-NN, Kernel SVM and Random Forests.

Logistic Regression

K-Nearest
Neighbors (K-NN)

Support Vector
Machine (SVM)

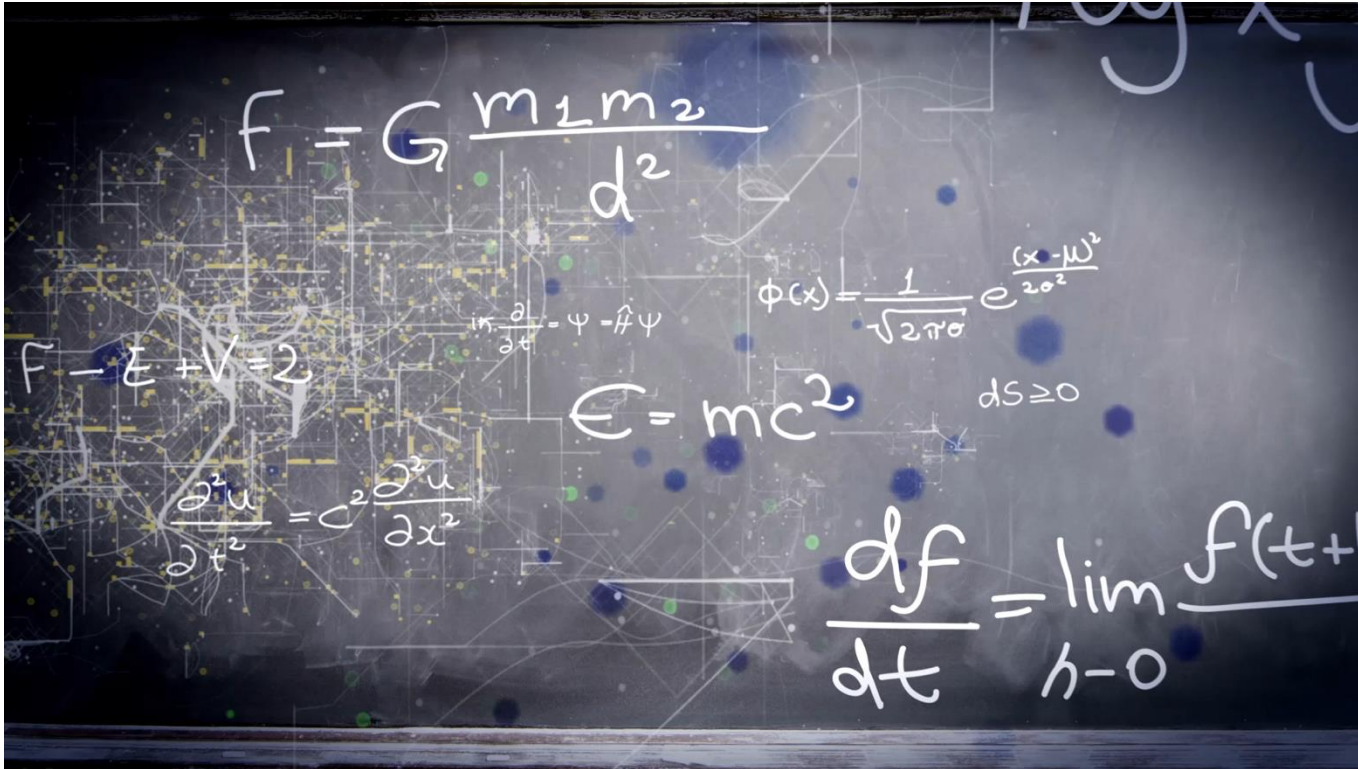
Kernel SVM

Naive Bayes

Decision Tree
Classification

Random Forest
Classification

CLASSIFICATION MODELS



K-NN CLASSIFIER

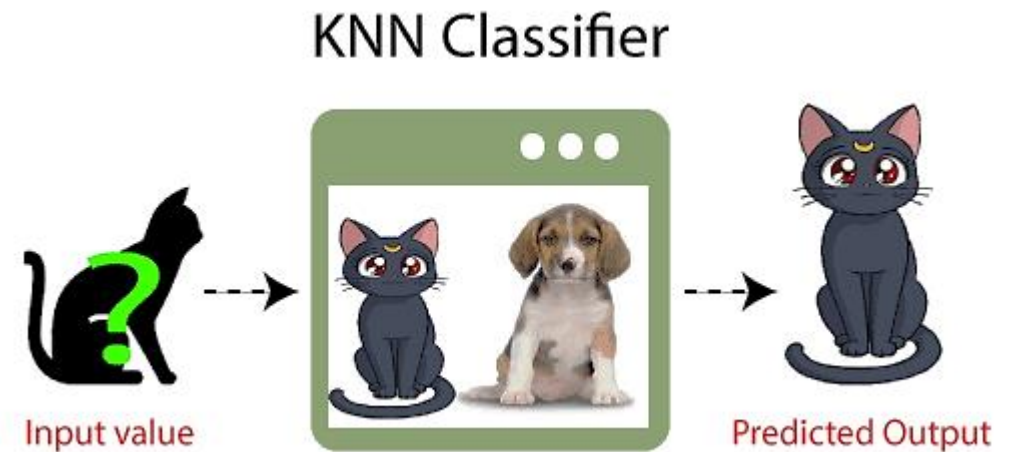
K-Nearest Neighbor (KNN)

INTRODUCTION TO KNN

- KNN is one of the most basic yet essential classification algorithms in machine learning.
- It belongs to the supervised learning domain and finds many applications in pattern recognition, data mining.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

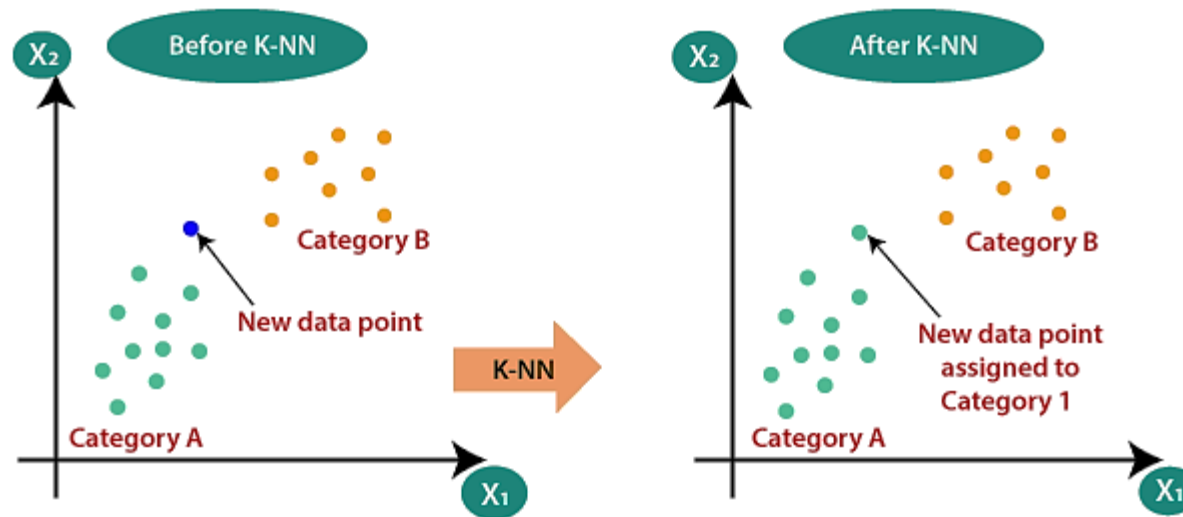
EXAMPLE

- Suppose, we have an image that looks similar to cat and dog,
- We want to know either it is a cat or dog.
- We can use the KNN algorithm, as it works on a similarity measure.
- KNN model will find the similar features of the new data set to the cats and dogs images.
- Based on the most similar features it will put it in either cat or dog category.



HOW DO WE NEED KNN

- Suppose there are two categories, i.e., Category A and Category B.
- we have a new data point x_1 , so this data point will lie in which of these categories.
- To solve this type of problem, we need a K-NN algorithm.



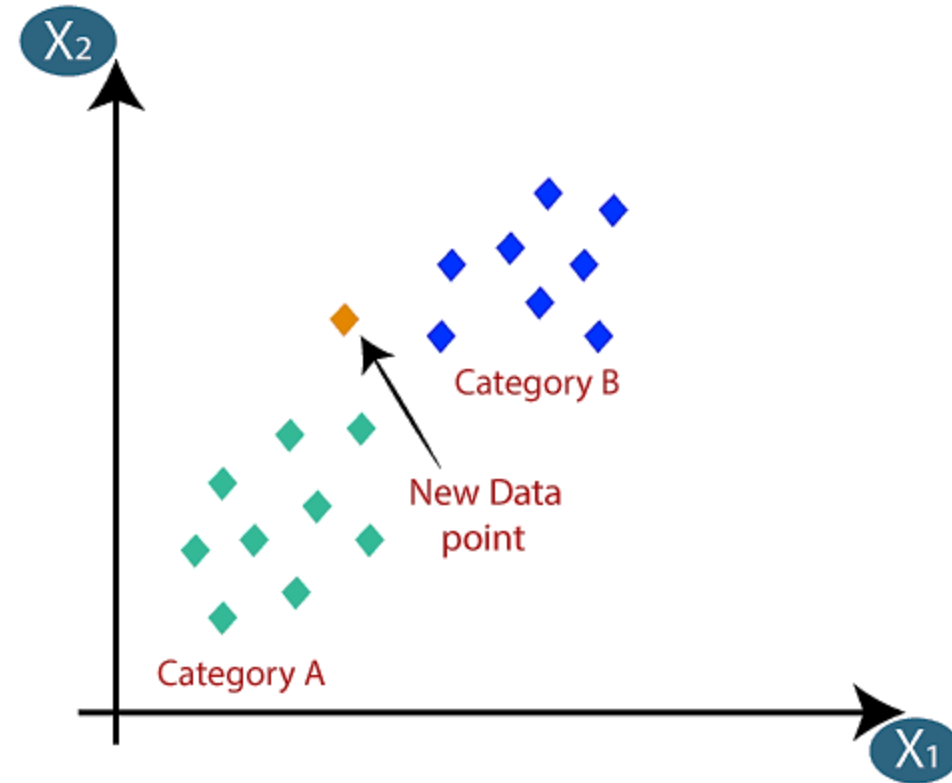
- With the help of K-NN, we can easily identify the category or class of a particular dataset.

HOW DOES K-NN WORK

- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of K number of neighbors
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** The model is ready.

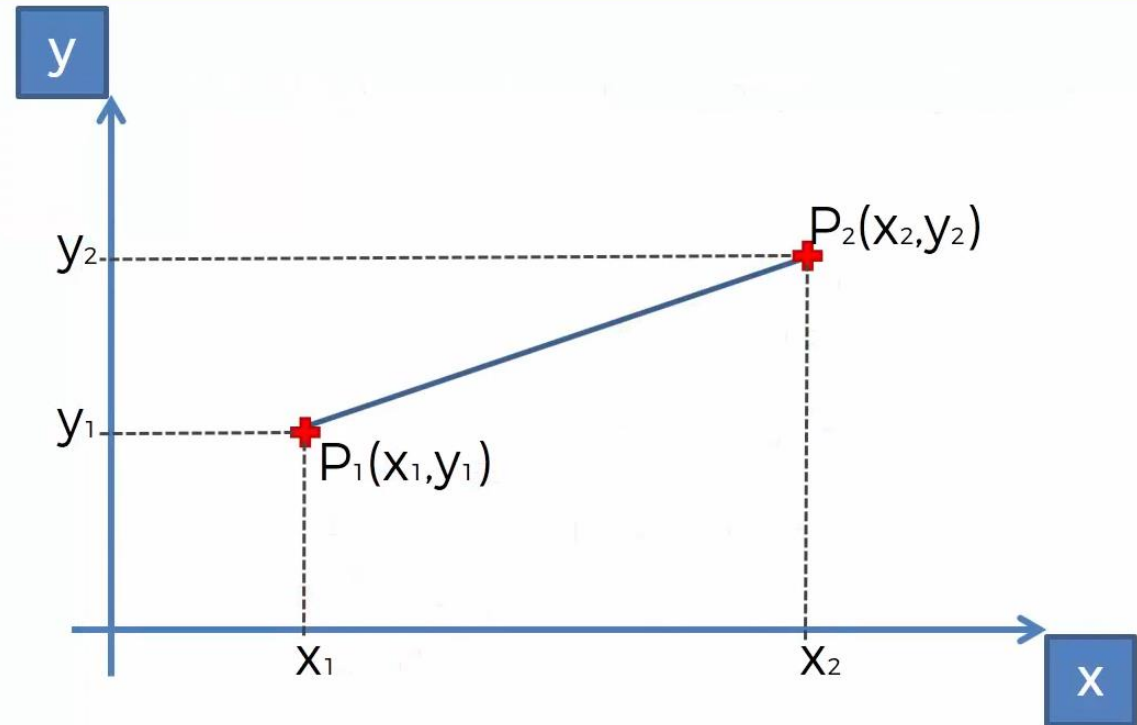
SOLVED EXAMPLE

- Suppose we have a new data point and we need to put it in the required category.
- Consider the image:
- Firstly, choose the number of neighbors, so we will choose $K=5$
- Next, calculate the Euclidean distance between the data points.
- Euclidean distance is the distance between two points.



EUCLIDEAN DISTANCE

- The equation below gives the Euclidean distance between points P_1 and P_2 .



$$\text{Euclidean Distance between } P_1 \text{ and } P_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

STEP 2: Take the $K = 5$ nearest neighbors of the new data point, according to the Euclidean distance



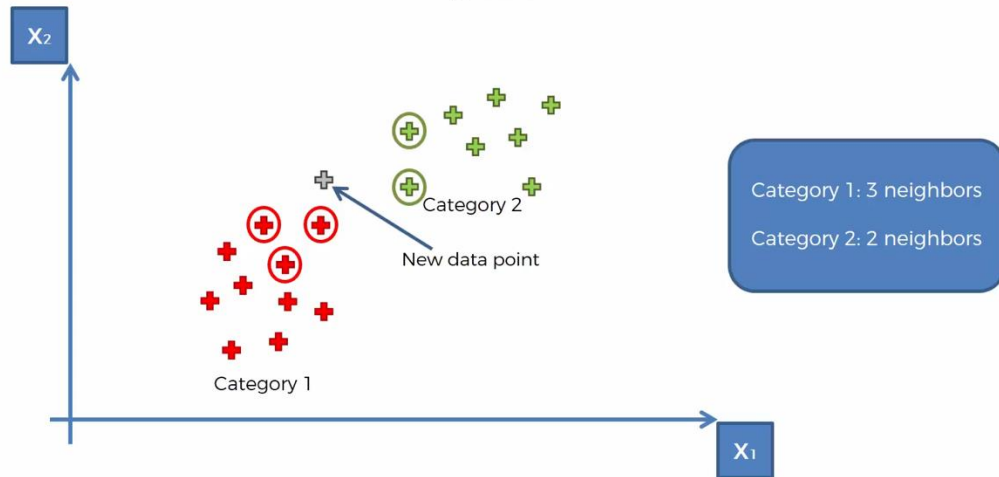
- Consider the number of data points closest to the new data point.

- Count the number of data points belonging to each category closest to the new data point

STEP 3: Among these K neighbors, count the number of data points in each category

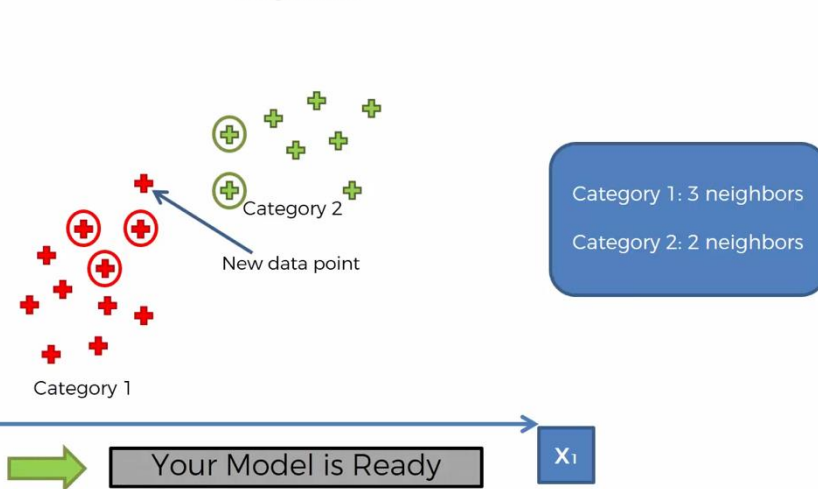


STEP 4: Assign the new data point to the category where you counted the most neighbors



- Category 1 has 3 neighbors
- Category 2 has 2 neighbors
- So we assign the new data point to category 1

STEP 4: Assign the new data point to the category where you counted the most neighbors



- The model is ready

HOW TO SELECT THE VALUE OF K

- There is no particular way to determine the best value for "K".
- We need to try some values randomly to find the best out of them.
- The most preferred value for K is 5.

PROS AND CONS OF KNN

PROS

- **Easy to implement** as the complexity of the algorithm is not that high.
- **Adapts Easily** – As per the working of the KNN algorithm it stores all the data in memory storage and hence whenever a new example or data point is added then the algorithm adjusts itself as per that new example and has its contribution to the future predictions as well.
- **Few Hyperparameters** – The only parameters which are required in the training of a KNN algorithm are the value of k and the choice of the distance metric (Euclidean, Hamming) which we would like to choose from our evaluation metric.

CONS

- **Does not scale** – As we have heard about this that the KNN algorithm is also considered a Lazy Algorithm. The main significance of this term is that this takes lots of data storage. This makes this algorithm both time-consuming and resource exhausting.
- **Curse of Dimensionality** the algorithm faces a hard time classifying the data points properly when the dimensionality is too high.
- **Overfitting issue**– As the algorithm is affected due to the curse of dimensionality it is prone to the problem of overfitting as well.

PYTHON IMPLEMENTATION

A vibrant green tree python with yellow markings, coiled against a black background. The snake's head is in the foreground, looking directly at the viewer. The text "PYTHON IMPLEMENTATION" is overlaid in white, serif capital letters across the middle of the image.

PROBLEM

The problem is same as before

You are given data by your manager of customers who have previously bought some older makes of your company's SUV. The data includes a total of 400 instances

Independent variables

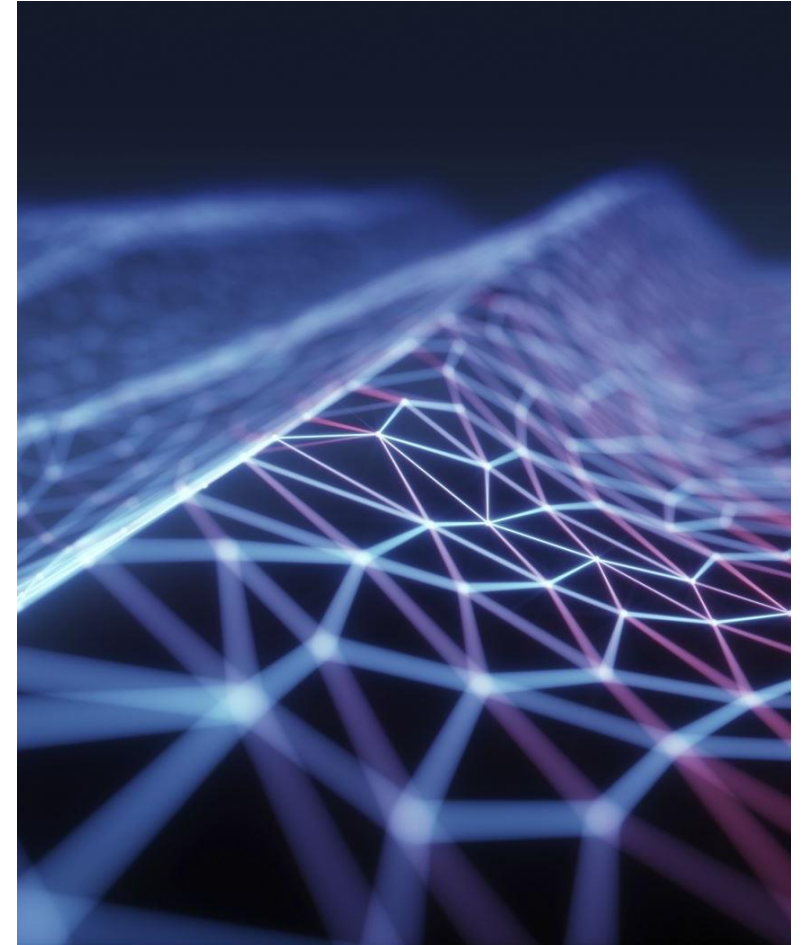
- Age
- Estimated salary

Dependent Variable

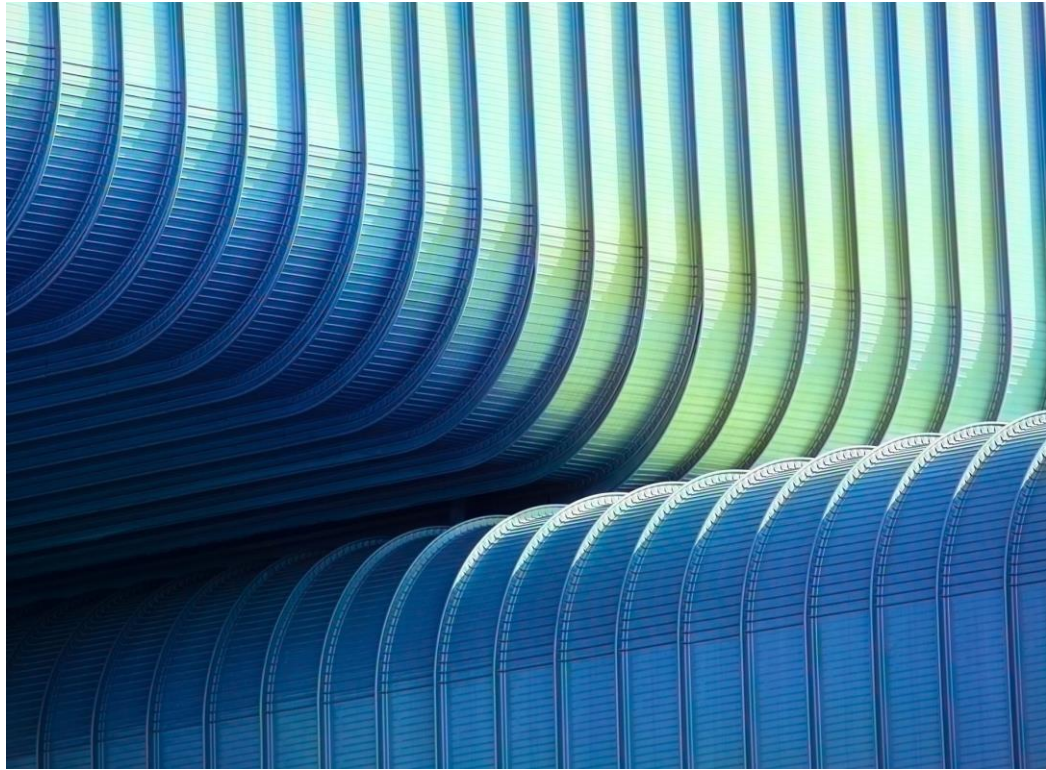
- Purchased (0 = no SUV purchased, 1 = SUV purchased)

Your company has just introduced a new SUV vehicle.

You are asked to predict who will buy the new SUV vehicle



STEPS



Import the
libraries

Import the
dataset

Split the dataset
into training and
testing samples

Feature scaling

Training the
model

Predicting the
new results

Predicting the
Test set results

Making the
confusion matrix

Visualize the
training set
results

Visualize the test
set results

IMPORTING LIBRARIES AND DATASET + DIVIDING THE DATASET INTO TRAIN AND TEST SETS

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv('Social_Network_Ads.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)
```

SCALING THE DATA

- We do not need to scale the dependent variable 'purchased' as it is already in the form of '0' and '1'.
- Remember scaling is always applied after splitting the dataset into test and train sets, because we want to avoid data leakage.

```
# Feature Scaling  
from sklearn.preprocessing import StandardScaler  
sc = StandardScaler()  
X_train = sc.fit_transform(X_train)  
X_test = sc.transform(X_test)
```


sklearn.neighbors: Nearest Neighbors

The `sklearn.neighbors` module implements the k-nearest neighbors algorithm.

User guide: See the [Nearest Neighbors](#) section for further details.

<code>neighbors.BallTree(X[, leaf_size, metric])</code>	BallTree for fast generalized N-point problems
<code>neighbors.DistanceMetric</code>	DistanceMetric class
<code>neighbors.KDTree(X[, leaf_size, metric])</code>	KDTree for fast generalized N-point problems
<code>neighbors.KernelDensity(*[, bandwidth, ...])</code>	Kernel Density Estimation.
<code>neighbors.KNeighborsClassifier(...)</code>	Classifier implementing the k-nearest neighbors vote.
<code>neighbors.KNeighborsRegressor([n_neighbors, ...])</code>	Regression based on k-nearest neighbors.
<code>neighbors.KNeighborsTransformer(*[, mode, ...])</code>	Transform X into a (weighted) graph of k nearest neighbors.
<code>neighbors.LocalOutlierFactor([n_neighbors, ...])</code>	Unsupervised Outlier Detection using the Local Outlier Factor (LOF).
<code>neighbors.RadiusNeighborsClassifier(...)</code>	Classifier implementing a vote among neighbors within a given radius.
<code>neighbors.RadiusNeighborsRegressor([radius, ...])</code>	Regression based on neighbors within a fixed radius.
<code>neighbors.RadiusNeighborsTransformer(*[, ...])</code>	Transform X into a (weighted) graph of neighbors nearer than a radius.
<code>neighbors.NearestCentroid([metric, ...])</code>	Nearest centroid classifier.
<code>neighbors.NearestNeighbors(*[, n_neighbors, ...])</code>	Unsupervised learner for implementing neighbor searches.
<code>neighbors.NeighborhoodComponentsAnalysis(...)</code>	Neighborhood Components Analysis.
< >	
<code>neighbors.kneighbors_graph(X, n_neighbors, *)</code>	Computes the (weighted) graph of k-Neighbors for points in X
<code>neighbors.radius_neighbors_graph(X, radius, *)</code>	Computes the (weighted) graph of Neighbors for points in X

<https://scikit-learn.org/stable/modules/classes.html#module-sklearn.neighbors>

sklearn.neighbors.KNeighborsClassifier

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *, weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=None)
```

[\[source\]](#)

Classifier implementing the k-nearest neighbors vote.

Read more in the [User Guide](#).

Parameters:

n_neighbors : int, default=5

Number of neighbors to use by default for [kneighbors](#) queries.

weights : {'uniform', 'distance'} or callable, default='uniform'

Weight function used in prediction. Possible values:

- 'uniform' : uniform weights. All points in each neighborhood are weighted equally.
- 'distance' : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
- [callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

algorithm : {'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'

Algorithm used to compute the nearest neighbors:

- 'ball_tree' will use [BallTree](#)
- 'kd_tree' will use [KDTree](#)
- 'brute' will use a brute-force search.
- 'auto' will attempt to decide the most appropriate algorithm based on the values passed to [fit](#) method.

leaf_size : int, default=30

Leaf size passed to BallTree or KDTree. This can affect the speed of the construction and query, as well as the memory required to store the tree. The optimal value depends on the nature of the problem.

p : int, default=2

Power parameter for the Minkowski metric. When $p = 1$, this is equivalent to using `manhattan_distance (l1)`, and `euclidean_distance (l2)` for $p = 2$. For arbitrary p , `minkowski_distance (l_p)` is used.

metric : str or callable, default='minkowski'

The distance metric to use for the tree. The default metric is `minkowski`, and with $p=2$ is equivalent to the standard Euclidean metric. See the documentation of [DistanceMetric](#) for a list of available metrics. If metric is "precomputed", X is assumed to be a distance matrix and must be square during fit. X may be a [sparse graph](#), in which case only "nonzero" elements may be considered neighbors.

Distance functions

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

Metrics intended for real-valued vector spaces:

identifier	class name	args	distance function
"euclidean"	EuclideanDistance	•	<code>sqrt(sum((x - y)^2))</code>
"manhattan"	ManhattanDistance	•	<code>sum(x - y)</code>
"chebyshev"	ChebyshevDistance	•	<code>max(x - y)</code>
"minkowski"	MinkowskiDistance	p	<code>sum(x - y ^p)^(1/p)</code>
"wminkowski"	WMinkowskiDistance	p, w	<code>sum(w * (x - y) ^p)^(1/p)</code>
"seuclidean"	SEuclideanDistance	V	<code>sqrt(sum((x - y)^2 / V))</code>
"mahalanobis"	MahalanobisDistance	V or VI	<code>sqrt((x - y)' V^-1 (x - y))</code>

TRAINING THE CLASSIFIER

- We will choose k i.e., `n_neighbors` as 5
- Distance metric as Euclidean distance i.e., Minkowski distance with $p=2$.

```
# training the classifier  
from sklearn.neighbors import KNeighborsClassifier  
classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)  
classifier.fit(X_train, y_train)
```

```
# Predicting a new result
```

```
print(classifier.predict(sc.transform([[30,87000]])))
```

Result

:

[0]

	0		0
0	0		0
1	0		0
2	0		0
3	0		0
4	0		0
5	0		0
6	0		0
7	1		1
8	0		0
9	1		0

Predicting the Test set results

```
y_pred = classifier.predict(X_test)
```

```
print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))
```

Making the Confusion Matrix

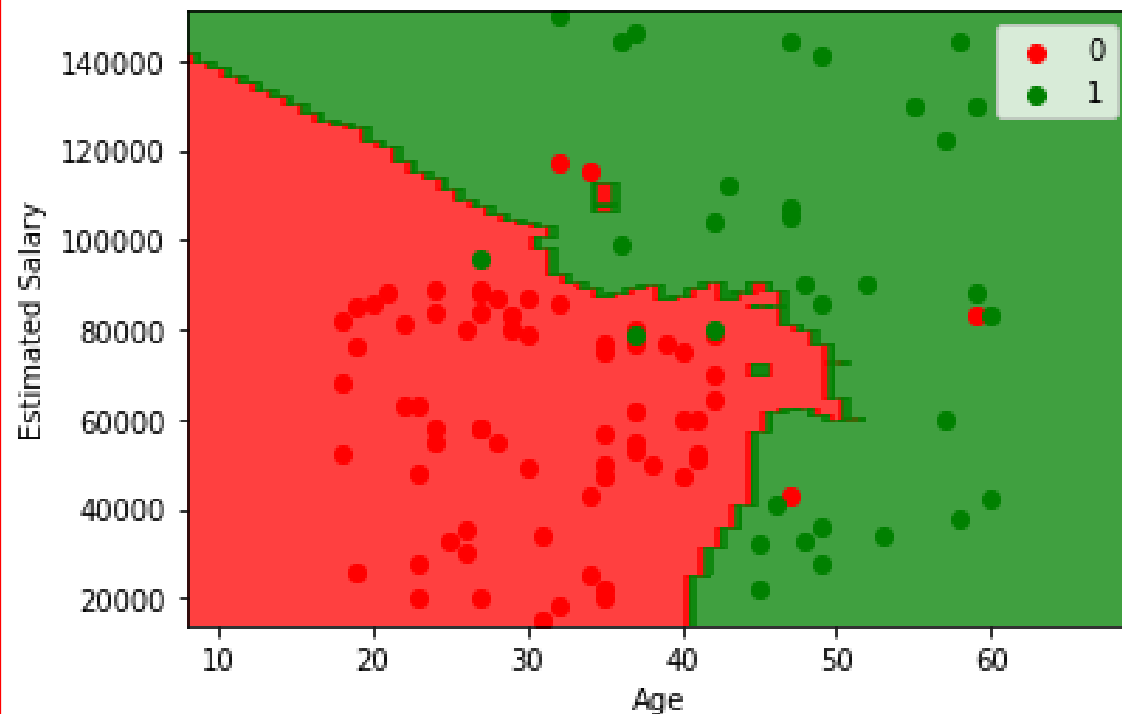
```
from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
print(cm)
accuracy_score(y_test, y_pred)
```

	0	1
0	64	4
1	3	29

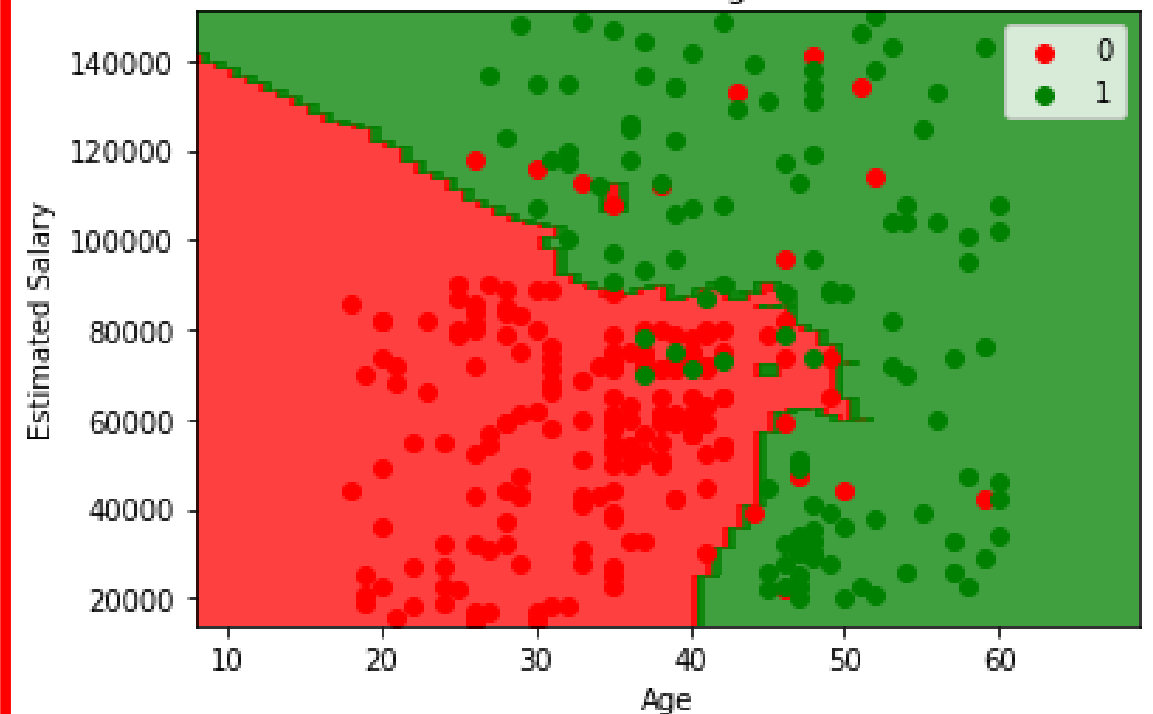
- The confusion matrix shows that 64 correct decisions for class 1 i.e., people who did not buy the SUV
- 29 correct classifications for the class 2 i.e., people who did not buy the SUV
- 4 incorrect decision about the people who bought the SUV but were classified as not buying the SUV
- 3 incorrect decisions about the people who did not buy the car but were classified as buying the car
- The accuracy is 93%
- $\text{Accuracy} = (\text{tp} + \text{tn}) / (\text{tp} + \text{tn} + \text{fp} + \text{fn})$

VISUALIZE THE TRAINING SET

K-NN (Test set)

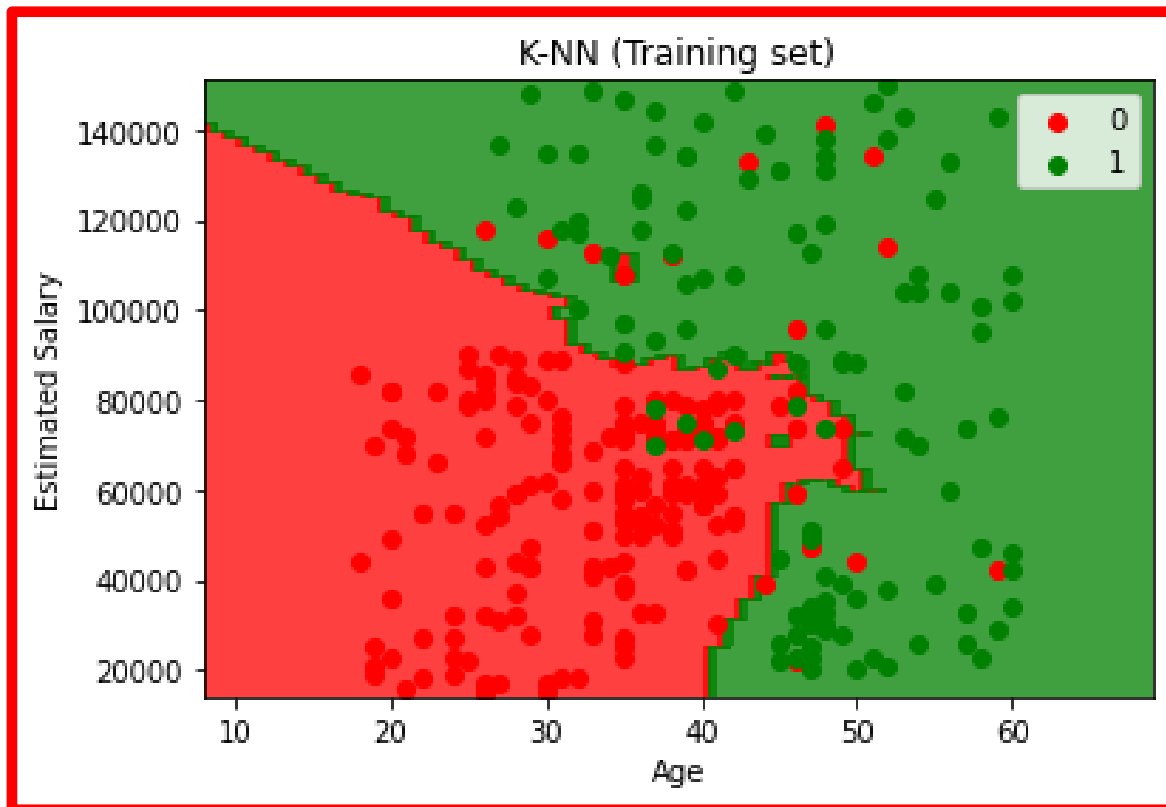


K-NN (Training set)



COMPARISON OF KNN WITH LOGISTIC REGRESSION

- It can be observed that the decision boundary for the logistic regression is a straight line
- In comparison the decision boundary for the



CONCLUSION

Logistic regression gave an accuracy of 89%

KNN gives an accuracy of 93%

We have improved the accuracy by 4%

K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data.

It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.

QUESTIONS

